

INF01151 – Sistemas Operacionais II

Aula 21 - Comunicação em grupo

Prof. Alberto Egon Schaeffer Filho

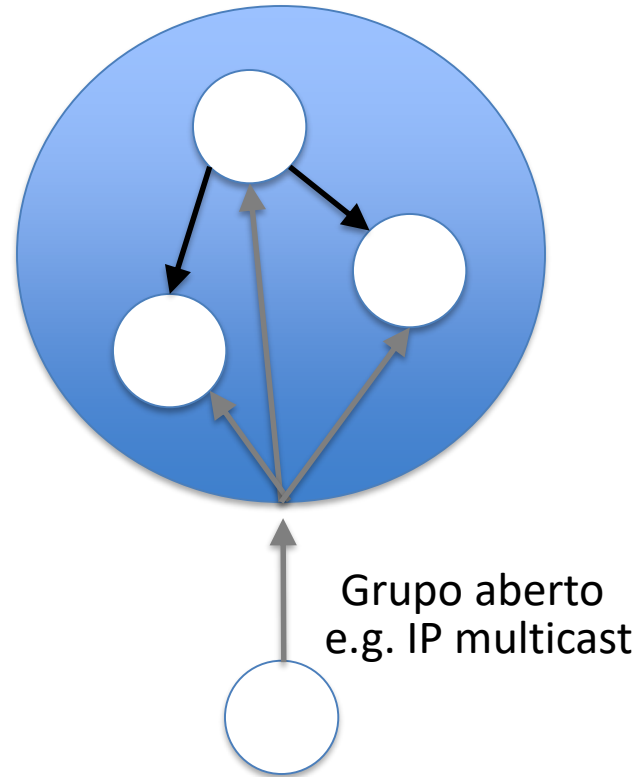
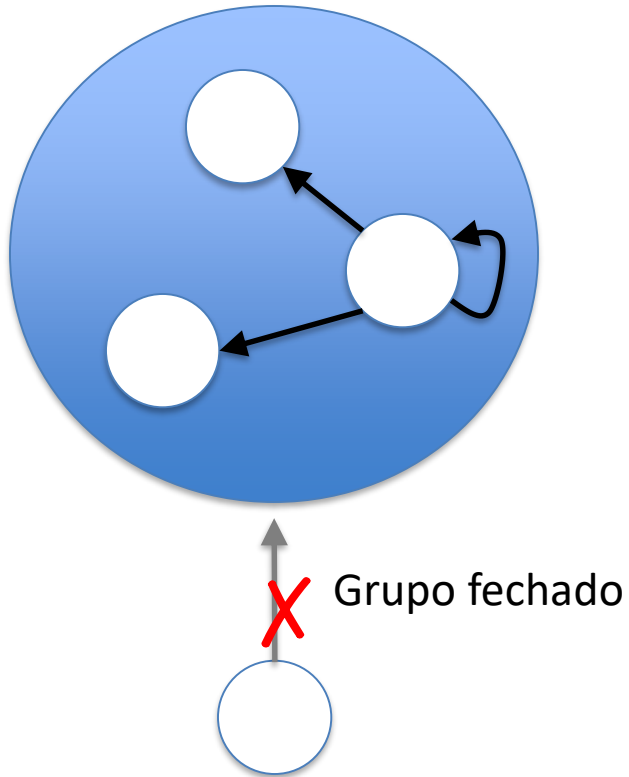
Introdução

- Forma elementar de comunicação entre processos é um-para-um
 - Um único emissor para um único receptor
 - Também denominada de ponto-a-ponto ou *unicast*
- Em muitos casos, comunicação em grupo é mais apropriada
 - Modelo de comunicação indireta
 - Mensagem enviada para o grupo deve ser entregue a todos os membros
 - Membership geralmente é transparente
- Algumas situações exigem comunicação entre grupo de processos
 1. Tolerância a falhas utilizando servidores replicados
 2. Localização de serviços
 3. Propagação de notificação de eventos

Comunicação em grupo

- Comunicação *um-para-muitos* (ou *multicast*)
 - Se processo é membro do grupo, ele pode receber mensagens enviadas para aquele grupo
 - Muito útil para encontrar recursos e divulgar informações
- Noção de grupo
 - Conjunto de processos que recebem as informações
 - Podem ser:
 - **Fechado**: apenas membros do grupo podem enviar mensagens
 - **Aberto**: qualquer processo pode enviar mensagens para o grupo
 - Questão de gerência
 - **Criação do grupo**: grupos permanentes existem mesmo quando não há membros
 - **Membership**: grupos tipicamente são dinâmicos, processos podem entrar e sair
 - **Overlapping ou não**: processos podem participar de número arbitrário de grupos

Grupo fechado vs. grupo aberto



Pontos genéricos de implementação

- Forma de implementar o *multicast*
 - Mensagem é enviada ao grupo, e entregue a todos os membros do grupo
 - Endereço *multicast* para identificar processos destino (ou melhor, grupo)
 - Emissor envia uma única mensagem para o grupo (ao invés de n operações de send)
 - Não somente uma conveniência para o programador, mas também mais eficiente
- *IP multicast* é um mecanismo de comunicação assíncrono
 - Em sua forma mais simples, não há como o emissor esperar por todos destinos estarem prontos para receber a mensagem
 - Emissor pode desconhecer quantos membros tem no grupo
 - Lado receptor:
 - Não bufferizado: destino perde o *multicast* se não está pronto para receber
 - Bufferizado: o destino receberá o *multicast* quando estiver pronto
- Outros aspectos: Ordenamento? Confiável?
 - Serão vistos na aula de hoje....

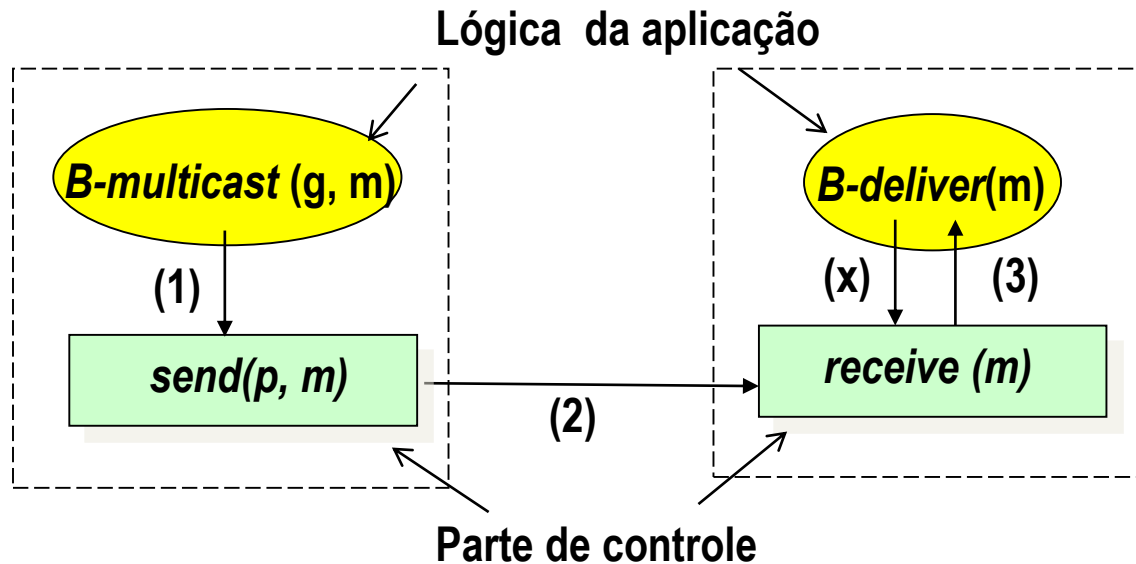
Confiabilidade em *multicast*

- Confiabilidade definida em termos de:
 - **Integridade**: mensagem recebida é a mesma que a enviada, s/ duplicações
 - **Validade**: mensagem enviada será eventualmente entregue
 - **Concordância**: se mensagem é entregue a um processo, então é entregue a todos do grupo
 - Propriedade “tudo ou nada”
 - Ou todos os processos do grupo recebem o *multicast* ou é como se nenhum tivesse recebido
 - Problema quando se prevê falhas emissor, receptor ou na rede
- Confirmações de entrega da mensagem *multicast*
 - **0-confiável**: emissor não espera nenhuma confirmação (*e.g.*, *IP multicast*)
 - **1-confiável**: emissor recebe a confirmação de um destino
 - **m-de-n confiável**: emissor espera a resposta de m processos de um grupo formado por n processos ($1 < m < n$)
 - **Confiável**: emissor espera confirmação de todos membros

Multicast básico

- Primitiva `multicast(g, m)` que garante que a mensagem será entregue ao destino
- Primitivas:
 - `B-multicast(g, m)`: para cada processo $p \in g$, `send(p, m)`
 - `B-deliver(m)`: *em* receive entrega m para $p \in g$

Confiável?



Multicast confiável sobre IP multicast

- Propriedades de multicast confiável
 - **Integridade**: processo correto $p \in g$ entrega mensagem m no máximo uma vez, tendo m sido enviada usando operação $multicast(g,m)$
 - **Validade**: se processo correto faz $multicast(g,m)$, então processo eventualmente entregará m (self-delivery)
 - **Concordância**: se algum processo correto entrega m , então todos os processos corretos $p \in g$ entregarão m em algum momento
 - *não é suportada pelo B-multicast do slide anterior
 - Relacionado à atomicidade (ou tudo ou nada)
- Multicast confiável pode ser implementado sobre IP multicast
 - **Assumption**: grupos fechados, e sem overlapping

Multicast confiável sobre IP multicast

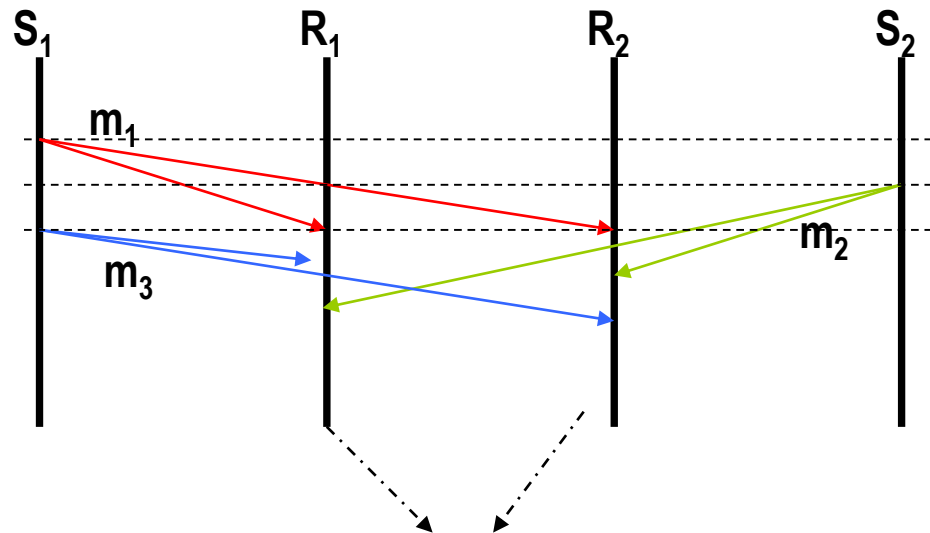
- Tipos de mensagens
 - **Piggybacked acks**: acks enviados junto com mensagem a ser transmitida
 - **Negative acks**: indicam a falta de uma mensagem esperada
- Cada processo $p \in g$:
 - S_g^p : número de mensagens enviadas por p
 - $\{R_g^q\}$: número de sequência da última mensagem de q para grupo g
- Quando p faz $R\text{-multicast}(g, m)$:
 - Envia junto S_g^p e acks $\langle q, R_g^q \rangle$ e depois incrementa S_g^p
 - Ack indicará o número de sequência da última mensagem enviada por q (vista por p)
- Quando processo recebe mensagem de q com número seq. S :
 - Se $S = R_g^q + 1$, processo faz $R\text{-deliver}(g, m)$
 - Se $S \leq R_g^q$, processo já entregou essa mensagem, então descarta
 - Se $S > R_g^q + 1$, ou se $ack(q) > R_g^q$, guarda m para depois ou solicita

Entrega ordenada de mensagens

- Multicast básico entrega mensagens em uma ordem arbitrária
 - Adequado para algumas aplicações: e.g., notificações no Facebook
 - Não satisfatório para muitas aplicações: e.g., eventos em usina nuclear
- Multicast ordenado
 - Todas as mensagens são entregues para todos os destinos em uma ordem que seja apropriada para a aplicação - semânticas possíveis:
 - **Ordenação FIFO:** preserva a ordem da perspectiva do emissor, ou seja, se processo envia uma m antes de m' , m será entregue antes de m' em todos os processos do grupo
 - **Ordenação causal:** se uma mensagem “*happens before*” outra mensagem, essa relação será preservada na entrega
 - **Ordenação total (consistente):** se m for entregue antes m' em um processo, então a mesma ordem será preservada em todos os processos

Ordenação FIFO

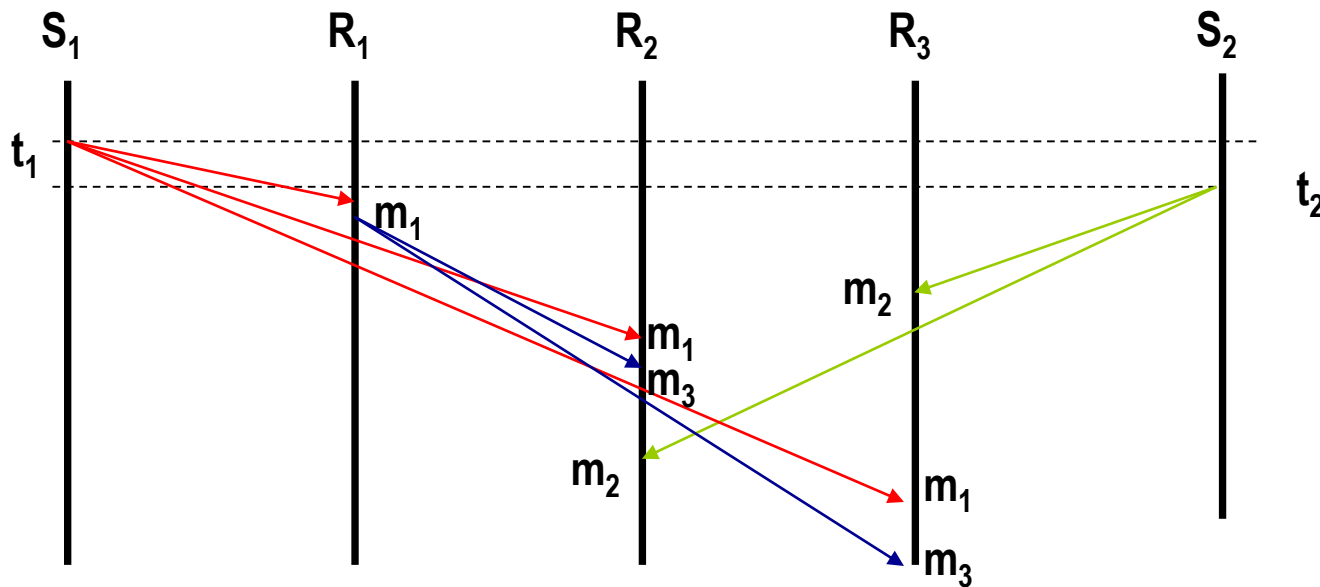
- Se um processo executa $multicast(g, m)$ e após $multicast(g, m')$, cada processo $q \in g$ realizará $deliver(m')$ após ter executado $deliver(m)$
 - Mensagens enviadas por um processo são entregues ao destino na ordem que foram enviadas



m_3 deve ser entregue depois de m_1 , mas ordem de m_2 não importa

Ordenação causal

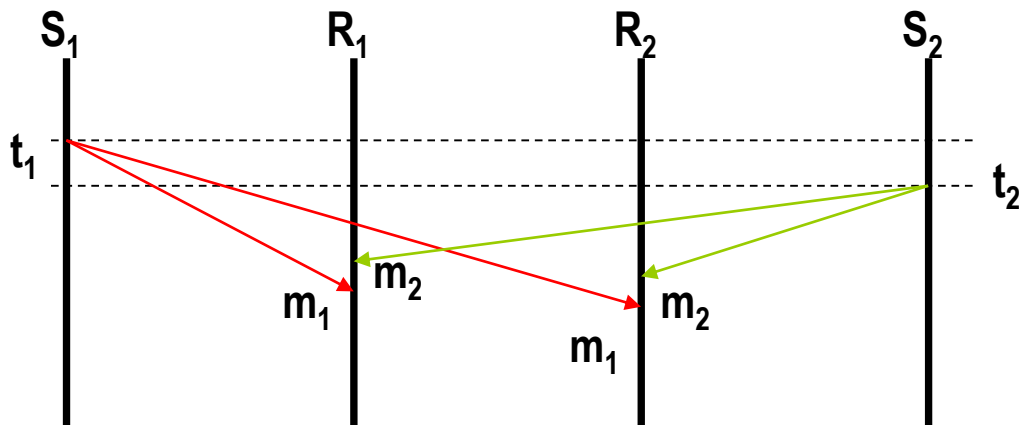
- Se $\text{multicast}(g, m) \rightarrow \text{multicast}(g, m')$ então $\text{deliver}(m')$ será executado após $\text{deliver}(m)$



Por causa da recepção de m_1 , R_1 envia a mensagem m_3 para R_2 e R_3 .
A ordem m_1 e m_3 deve ser mantida nesses dois destinos.

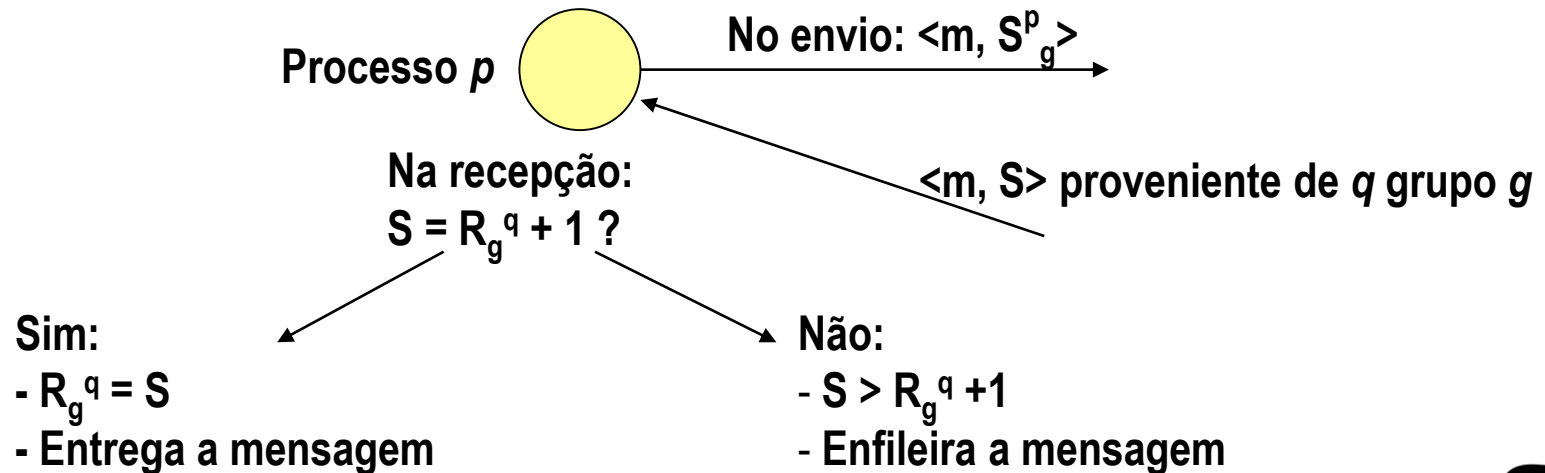
Ordenação total (consistente)

- Se um processo $p \in g$ executa $deliver(m)$ antes de $deliver(m')$, então qualquer outro processo $q \in g$ entregará m' após m
 - Garantia que todas as mensagens são entregues na mesma ordem em todos os processos mesmo que difira da ordem de emissão



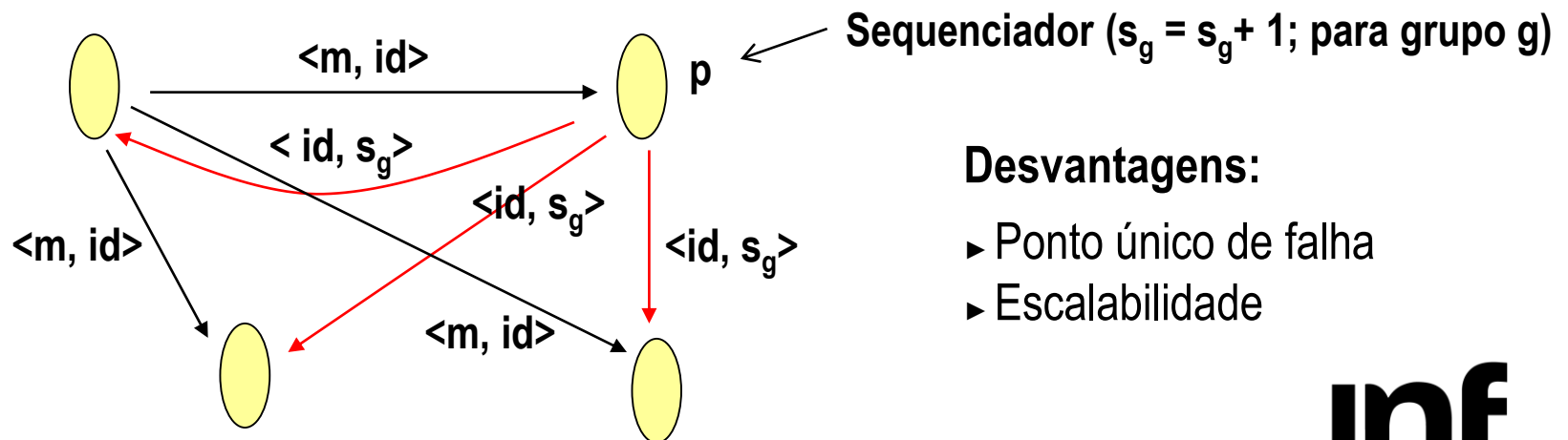
Algoritmo para ordenação FIFO

- Todas as msgs de um emissor são entregues na mesma ordem
- Baseado em números de seqüência
- Dois contadores em cada processo:
 - S_g^p : número de mensagens que processo p enviou a g
 - $\{R_g^q\}$: número de seqüência da última mensagem recebida do processo q



Algoritmo ordenação total (centralizado)

- Idéia básica: atribuir identificadores totalmente ordenados a mensagens de multicast
 - Para que cada processo tome a mesma decisão quanto ao ordenamento
- Definição de um processo $p \in g$ como **sequenciador**
 - Número de sequência por grupo, e não por processo
 - Recebe mensagem de um processo q e a retransmite a todos os membros com um identificador único sequencial



Algoritmo ordenação total (centralizado)

1. Algorithm for group member p

On initialization: $r_g := 0$;

To TO-multicast message m to group g

B-multicast($g \cup \{\text{sequencer}(g)\}$, $\langle m, i \rangle$);

On B-deliver($\langle m, i \rangle$) with $g = \text{group}(m)$

Place $\langle m, i \rangle$ in hold-back queue;

On B-deliver($m_{\text{order}} = \langle \text{"order"}, i, S \rangle$) with $g = \text{group}(m_{\text{order}})$

wait until $\langle m, i \rangle$ in hold-back queue and $S = r_g$;

TO-deliver m ; // (after deleting it from the hold-back queue)

$r_g = S + 1$;

2. Algorithm for sequencer of g

On initialization: $s_g := 0$;

On B-deliver($\langle m, i \rangle$) with $g = \text{group}(m)$

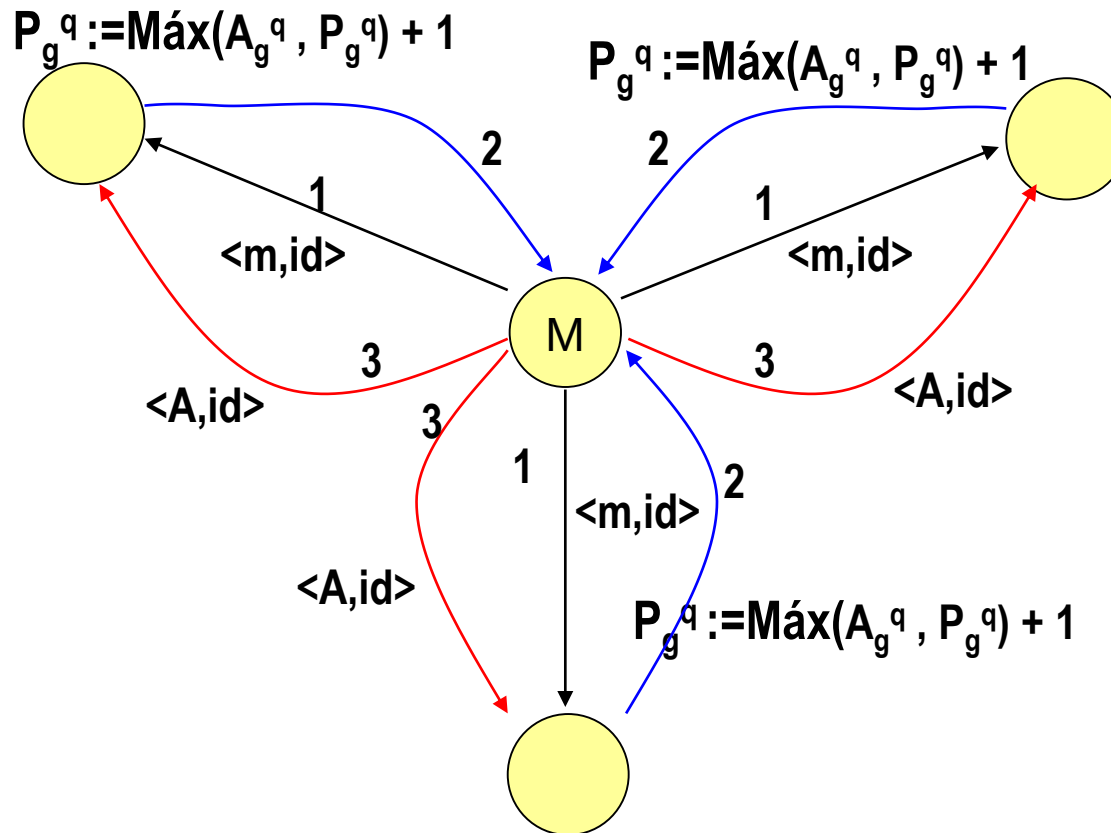
B-multicast(g , $\langle \text{"order"}, i, s_g \rangle$);

$s_g := s_g + 1$;

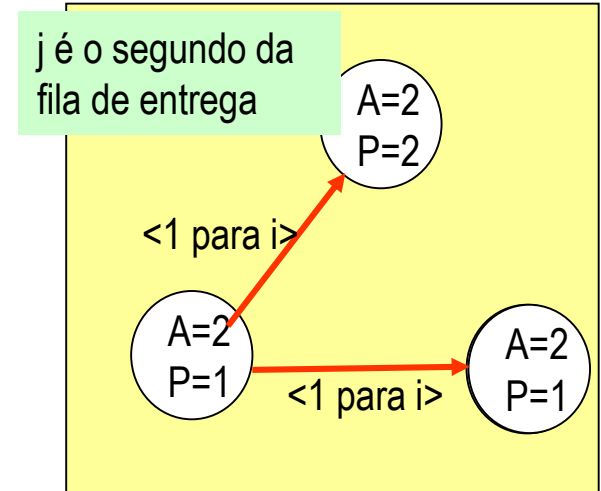
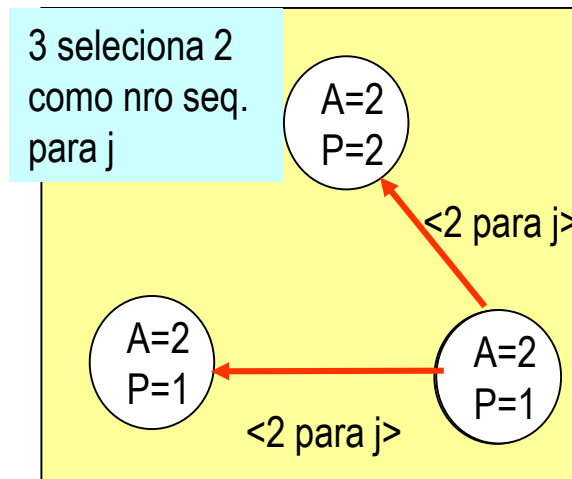
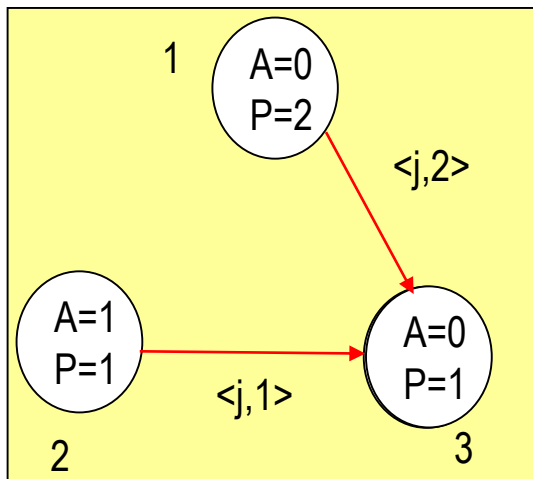
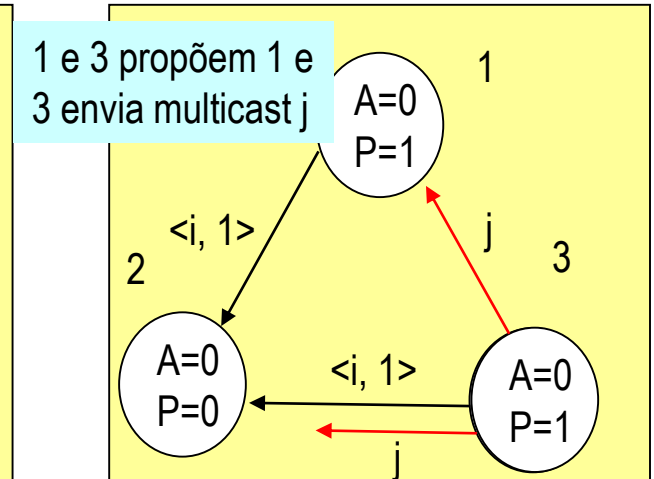
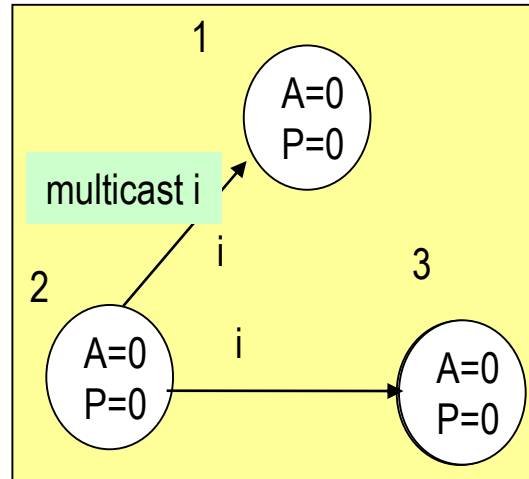
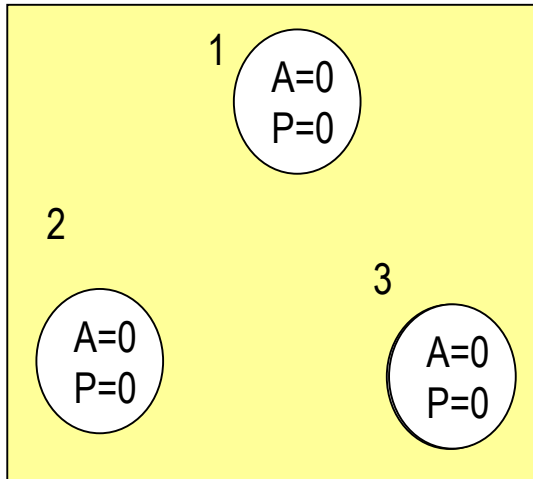
Algoritmo ordenação total (distribuído)

- Ideia básica:
 - Processos devem coletivamente concordarem na atribuição de números de sequência
 - Ao receber uma mensagem, cada processo *propõe* um número de seq.
 - Cada processo q no grupo g armazena o maior número de sequência acordado até o momento (A^q_g), e seu maior número proposto P^q_g
- Funcionamento:
 - Processo p envia $\langle m, id \rangle$ para g (id é um identificador de m)
 - Cada processo q responde a p propondo um número de sequência $P^q_g := \max(A^q_g, P^q_g) + 1$, que é enviado para o emissor
 - Emissor seleciona o maior número de sequência proposto pelos membros e envia uma nova mensagem $\langle A, id \rangle$ com esse número de sequência
 - Membros associam número de sequência a mensagem id
 - Membros realizam entrega baseado no num. de sequência

Algoritmo ordenação total (distribuído)



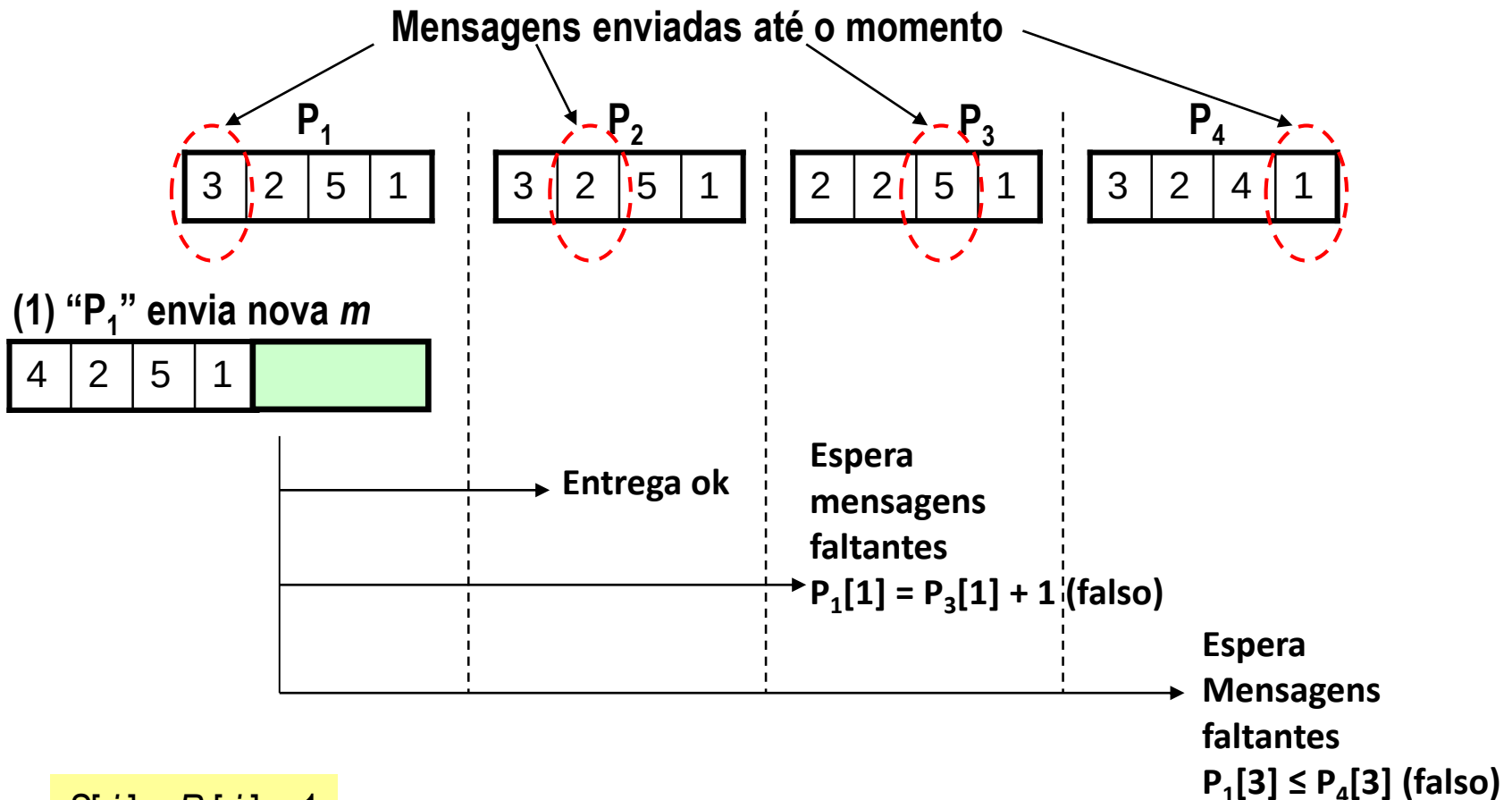
Exemplo: ordenação total



Algoritmo ordenação causal

- Baseado em relógios vetoriais (lógicos)
 - Relação causal apenas em termos de multicast
 - Cada entrada contém n° de multicasts que “*happened before*” próximo multicast
- Funcionamento:
 - Cada membro mantém um vetor de n elementos
 - Membro i corresponde a i -ésima entrada do vetor
 - Valor da entrada é o n° de sequência da última mensagem recebida de i
 - Ao enviar uma mensagem, cada membro incrementa sua própria entrada no vetor e envia o vetor junto com a mensagem
 - Entrega de mensagem enviada por P_i é feita no destino quando (simultaneamente):
 1. $S[i] = R[i] + 1 \rightarrow$ destino não perdeu nenhuma mensagem do remetente
 2. $S[j] \leq R[j]$ para todo $j \neq i \rightarrow$ destino já entregou qualquer mensagem que remetente tenha entregue quando ele fez o multicast
 - onde S é vetor recebido junto a mensagem, R é o vetor local

Algoritmo ordenação causal (*cont.*)



$$S[i] = R[i] + 1$$

$$S[j] \leq R[j]$$

Algoritmo ordenação causal

Algorithm for group member p_i ($i = 1, 2, \dots, N$)

On initialization

$V_i^g[j] := 0$ ($j = 1, 2, \dots, N$);

To CO-multicast message m to group g

$V_i^g[i] := V_i^g[i] + 1$;

$B\text{-multicast}(g, \langle V_i^g, m \rangle)$;

On $B\text{-deliver}(\langle V_j^g, m \rangle)$ from p_j , with $g = \text{group}(m)$

place $\langle V_j^g, m \rangle$ in hold-back queue;

wait until $V_j^g[j] = V_i^g[j] + 1$ and $V_j^g[k] \leq V_i^g[k]$ ($k \neq j$);

$CO\text{-deliver } m$; // after removing it from the hold-back queue

$V_i^g[j] := V_i^g[j] + 1$;

Comentário final: *multicast* na Internet

- Existe apenas na camada de rede
 - Endereço IP classe D (224.0.0.0 a 239.255.255.255)
 - IP realiza entrega computador a computador
 - Portas identificam processos origem e destino
- Noção de grupo é dinâmica
 - Membros entram e saem sem aviso prévio
 - Não há garantias de ordem, nem de entrega
 - Multicast 0-confiável
 - Podem participar de vários grupos simultaneamente
 - Grupo aberto
- Disponível apenas para sockets UDP (DGRAM)

 Reforça a necessidade dos algoritmos e das semânticas estudadas

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 4 (seção 4.4)
 - Capítulo 6 (seção 6.2)
 - Capítulo 15 (seção 15.4)